
THU
Technische
Hochschule
Ulm


HENSOLDT

Plugin System’s in Rust

Gabriel Zimmermann

I. Abstract

Somethign something what approaches are there to setup plugin systems in Rust. What are the pros and cons…

II. Metadata University…

Informatik
Bachelor thesis at Ulm University of Applied Sciences
Department of Computer Science
Degree Program Informatik
Presented by Gabriel Zimmermann
April 2026
1) Reviewer: Prof. Dr.-Ing. Klaus Baer
2) Reviewer: Prof. Dr.-Ing. Philipp Graf

III. Introduction

Analysis of dynamic plug-in systems in rust. Comparison of different approaches focusing the pros and cons.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri. [1]

IV. Overview

Problem: Rust has unstable ABI, which makes the same approach of C not functional.

Therefore, this paper looks at various approaches.

This can be seperated into static and dynamic approaches. Static requires the recompilation of binaries.

A. Goal

Analysis of these things

a) *Static:*

I can only mention here, that we can enable/disable parts of code during compile time, yes? aka #[cfg(feature = “enable_insane_feature”)] or cfg!(feature = “enable_insane_feature”) -> extra careful during development, that all required features are structered without accidental deactivations??

b) *Dynamic:*

- C ABI
- unstable Rust ABI
- “stable” Rust ABI Crate
- libloading -> can be used for c and rust (but both un- and stable rust ABI should work here :3)
- rust bridge

I should probably explain #[no_mangle] …

Mention speed up compile times?

B. Baseline

Simple data fusion of N temperature datas (2x float 64bits).

- Average
- First data field

V. Methods

Goal of the research was to analyse the following points:

- performance
- development complexities
- limitations
- safety
- interoperability

A. Performance

Only “pure” quantitative measurement will be the performance and maybe safety (using safety levels).

a) *Tools:*

Using criterion and gungraun benchmarks. Not a perfect measurement, but it should give us enough hints about what happens behind the scenes.

Using black_box, we can ensure that the iternal code isn’t hyper-optimized by the compiler, which can lead to more accurate benchmarks.

Gungruan uses valgrind internally.

b) *Benchmarks:*

To test all of this, we use a simple temperature simulation, where an random amount of sensors will pick up the data.

Benches are run with the follolwing set of configurations:

- run app “regarulary” with black_box
- run average temperature fusion code with black_box once (sensors: 1 vs. 1000000)
- run average temperature fusion code without black_box (sensors: 1 vs. 1000000)

c) *Evaluation:*

Gunguan helps us see the internal required instructions on “my” CPU. This result should correlate with the actual run time using criterion. We can then reason about the expected overhead of a choosen approad.

B. Development complexities

Subjective, but also Qualitative Meaurement

How will this impact a development team? This might just be an extra Evaluation of Limitations

C. Limitations

Might be part of “complexities”??

Qualitative Meaurement

D. Safety

Atemmpting Quantative by using safety levels.

Rust Lanuage generally very safe. Errors often need to be “forced” or careless usage of unwinding. [2]

E. interoperability

How flexible are the presented approaches? Can they be linked with other libraries?

References

[1] L. Morris and R. Astley, “Net Wok++: Taking distributed dumplings to the cloud,” *Armenian Journal of Proceedings*, vol. 65, pp. 101–118, 2022.

[2] T. R. P. Developers, “The Rustonomicon: Unwinding.” Accessed: Feb. 24, 2026. [Online]. Available: https://doc.rust-lang.org/nomicon/unwinding.html#unwinding